5.1 Modelo arquitectónico

Proyecto: Monitor de Consumo Eléctrico — EnergyMonitor **Ficha:** 3145556 — Análisis y Desarrollo de Software (ADSO), SENA CIES Regional Huila **Versión:** 1.0

> **Nota de actualización (27/10/2026).** Este documento describe el diseño de la versión 1.0, cuando el backend era el monolito modular `energy-monitor-backend` (Spring Modulith). Ese diseño evolucionó: los siete contextos acotados son hoy **siete microservicios** detrás del API gateway `en-monitor-api-gateway` (8080) — seguridad 8082, dispositivos 8084, mediciones 8085, hogares 8086, notificaciones 8087, alertas 8088 y recomendaciones 8089 —, comunicados por REST interno (`/internal/**`) y eventos de RabbitMQ con outbox transaccional, sobre el mismo esquema MySQL compartido (37 tablas). El backend modular quedó retirado. Las capas hexagonales, los contextos y los patrones descritos aquí se conservan dentro de cada servicio; la arquitectura vigente, las rutas, los eventos y la configuración están en el manual técnico (secciones 9 a 13) y el modelo de datos vigente en `06-base-datos`.

1. Propósito

Esta sección define la arquitectura del sistema: el estilo arquitectónico elegido, la organización de sus capas, la separación en contextos acotados, los componentes que lo integran y los protocols de comunicación entre ellos.

El documento responde a cuatro preguntas:

1. **¿Cómo se organiza el sistema?** — Estilo arquitectónico y capas.
2. **¿De qué componentes se compone?** — Vista de componentes y tecnologías.
3. **¿Cómo se comunican los componentes?** — Vista de despliegue y protocolos.
4. **¿Qué restricts el diseño?** — Atributos de calidad y reglas arquitectónicas verificables.

2. Estilo arquitectónico

2.1 Definición adoptada

El sistema adopta una arquitectura **cliente-servidor de tres capas**, combinada con:

Estilo	Ámbito	Justificación
Tres capas (presentación – lógica – datos)	Sistema completo	Separa la interfaz de usuario del negocio y de la persistencia, permitiendo evolucionar cada capa de forma independiente
Monolito modular (`Spring Modulith`)	Backend	Un único despliegue, pero con límites de módulo verificados automáticamente por compilación de pruebas
Arquitectura hexagonal (puertos y adaptadores)	Dentro de cada módulo	El dominio no depende de nada externo: las tecnología se conectan mediante adaptadores intercambiables
API REST sin estado	Contrato entre front-ends y backend	Permite que web y móvil compartan exactamente el mismo contrato
Cliente-publish / Oyente	Entre contextos del backend	Desacopla la producción de mediciones de la generación de alertas y recomendaciones

> **Por qué un monolito modular y no microservicios.** El volumen de datos y el número de usuarios del proyecto no justifican el costo operativo de una distribución separada en servicios independientes. El monolito modular ofrece la mayor parte de sus beneficios —despliegue único, transacciones locales, equipos autónomos dentro del código— sin introducir el acoplamiento de red ni la complejidad de la consistencia

> distribuida. La separación en contextos queda registrada en el código y verificada por pruebas, por lo que una extracción futura es viable módulo a módulo.

2.2 Arquitectura de tres capas

Figura 3. *Arquitectura general del sistema en tres capas.*

Capa	Tecnología	Responsabilidad	Ubicación del código
Presentación	React 19 (web) · Expo SDK 57 (móvil)	Interfaz de usuario, validación de formularios, manejo de sesión y consumo de la API REST	`Energy-Monitor-application/src` · `Energy-Monitor-Mobile/src`
Lógica	Java 21 + Spring Boot 4.1.1 + Spring Modulith 2.1.1	Autenticación, autorización, casos de uso, validación de reglas de negocio y publicación de eventos	`energy-monitor-backend/src/main/java/com/energymonitor`
Datos	MySQL 8.4 + Spring Data JPA + Liquibase	Persistencia de las 29 tablas, esquema versionado y consultas agregadas	`src/main/resources/db/changelog`
Transversal	Docker Compose · Mosquitto (MQTT) · Mailpit · Actuator	Base de datos, transporte de mediciones, correo de pruebas y verificación de salud	`docker-compose.yml` · `application.yaml`

3. Vista de componentes

3.1 Backend — siete contextos acotados

El backend es una aplicación Spring Boot organizada en **siete contextos acotados** (*bounded contexts*). Cada uno declara explícitamente sus dependencias permitidas en su archivo `package-info.java`.

Figura 4. *Mapa de contextos del backend y sus dependencias permitidas.*

Contexto	Responsabilidad	Dependencias permitidas	Estado
`security`	Registro, autenticación, sesiones, contraseñas, perfil, RBAC, auditoría	Ninguna	Activo
`home`	Hogares, miembros, favoritos, código de acceso, umbrales	`security::api`	Activo
`device`	Dispositivos medidores, API key, estado de conexión, aprovisionamiento	`home::api`	Activo
`measurement`	Registro y análisis de mediciones, niveles de consumo, estadísticas	`device::api`, `home::api`	Activo
`alert`	Generación automática de alertas y bandeja de consulta y resolución	`home::api`, `device::api`, `measurement::api`	Activo
`recommendation`	Recomendaciones de ahorro mediante análisis del consumo	`device::api`, `measurement::api`, `home::api`	Activo
`notification`	Envío de correo y Web Push, registro de entregas	`security::api`, `home::api`, `device::api`, `alert::api`, `recommendation::api`	Activo

> **Los límites son ejecutables, no documentales.** La clase `ModularityTests` invoca `ApplicationModules.verify()`, que analiza las clases compiladas y falla si algún módulo importa una clase interna de otro módulo. La regla arquitectónica no depende de la disciplina del equipo: si alguien la incumple, la construcción falla.

3.2 Contrato entre contextos

Cada contexto expone un contrato público en su subpaquete `api/`, declarado con la anotación `@NamedInterface`:

Interfaz pública	Módulo	Propósito
`UserProfileQuery`, `PasswordResetDeliveryPort`, `EmailVerificationDeliveryPort`	`security::api`	Perfil, correo de recuperación y verificación
`HomeApi`, `HomeThresholdsDto`	`home::api`	Consulta de hogares y umbrales desde otros contextos
`DeviceApi`, eventos `DeviceLinked`, `DeviceUnlinked`, `DeviceConnectivityLost`, `DeviceConnectivityRestored`	`device::api`	Consulta de dispositivos y publicación de eventos de conectividad
`MeasurementApi`, `MeasurementRecorded`, `ConsumptionLevelDto`, `HourlyEnergyDto`, `RiskConsumption`	`measurement::api`	Consulta de mediciones y publicación del evento central
`AlertApi`, `AlertRaised`, `AlertType`, `AlertStatus`	`alert::api`	Consulta de alertas y publicación de alertas generadas
`RecommendationCreated`, `RecommendationType`, `RecommendationStatus`	`recommendation::api`	Publicación de recomendaciones generadas
(sin interfaz pública)	`notification`	Solo publica mensajes; no expone contrato

Figura 5. *Flujo de dependencias entre contextos: sólo a través de interfaces públicas y eventos.*

3.3 Estructura interna de un contexto

Todos los módulos repiten la misma organización, derivada de la arquitectura hexagonal:

```
<módulo>/
├── package-info.java        @ApplicationModule(allowedDependencies = {...})
├── api/                     @NamedInterface — contrato público hacia otros módulos
├── domain/model/            Agregados, objetos de valor, enumeraciones, Preconditions
├── application/
│   ├── port/in/             Interfaces de caso de uso (puerto de ENTRADA)
│   ├── port/out/            Interfaces de persistencia y servicios (puerto de SALIDA)
│   ├── command/             Records de entrada de los casos de uso
│   ├── result/              Records de salida de los casos de uso
│   ├── usecase/             Implementaciones de los puertos de entrada
│   └── exception/           Excepciones de aplicación
├── adapter/
│   ├── in/web/              @RestController, dto/, manejadores de excepción
│   ├── in/event/            Oyentes (@EventListener)
│   ├── in/mqtt/             Consumidores MQTT
│   ├── in/scheduler/        Tareas programadas (@Scheduled)
│   ├── out/persistence/     Entidades JPA, repositorios, mapeadores, adaptadores
│   ├── out/security/        Hashers, generadores
│   ├── out/client/          Adaptadores hacia la API de otro módulo
│   ├── out/event/           Publicadores de eventos
│   ├── out/generator/       Generadores de identificadores
│   ├── out/mqtt/            Publicación de comandos al dispositivo
│   └── out/push/            Web Push
└── infrastructure/          @Configuration y orquestación de Spring
```

3.4 Puertos y adaptadores

Un **puerto** es una interfaz que declara lo que el módulo necesita; un **adaptador** es la implementación concreta que conecta ese puerto con una tecnología.

Dirección	Ubicación	Ejemplo de puerto	Ejemplo de adaptador
Puerto de entrada	`application/port/in`	`RegisterUser`, `CreateHome`, `RegisterMeasurement`	`RegisterUserService`, `CreateHomeService`
Puerto de salida	`application/port/out`	`UserPersistencePort`, `PasswordHasherPort`	`UserPersistenceAdapter`, `BCryptPasswordHasherAdapter`

Volumen de puertos en el sistema:

Módulo	Puertos de entrada	Puertos de salida
`security`	17	23
`home`	12	8
`device`	10	9
`measurement`	7	6
`alert`	10	7
`recommendation`	3	5
`notification`	5	6
Total	**64**	**64**

> **Qué se compra con esta estructura.** Gracias a los puertos y adaptadores, las pruebas de aplicación pueden ejercitar los casos de uso con dobles de prueba sin levantar Spring ni MySQL, y las tecnologías criptográficas, de correo o de persistencia pueden sustituirse sin tocar una sola línea de dominio.

4. Arquitectura de eventos entre contextos

4.1 Publicación y escucha

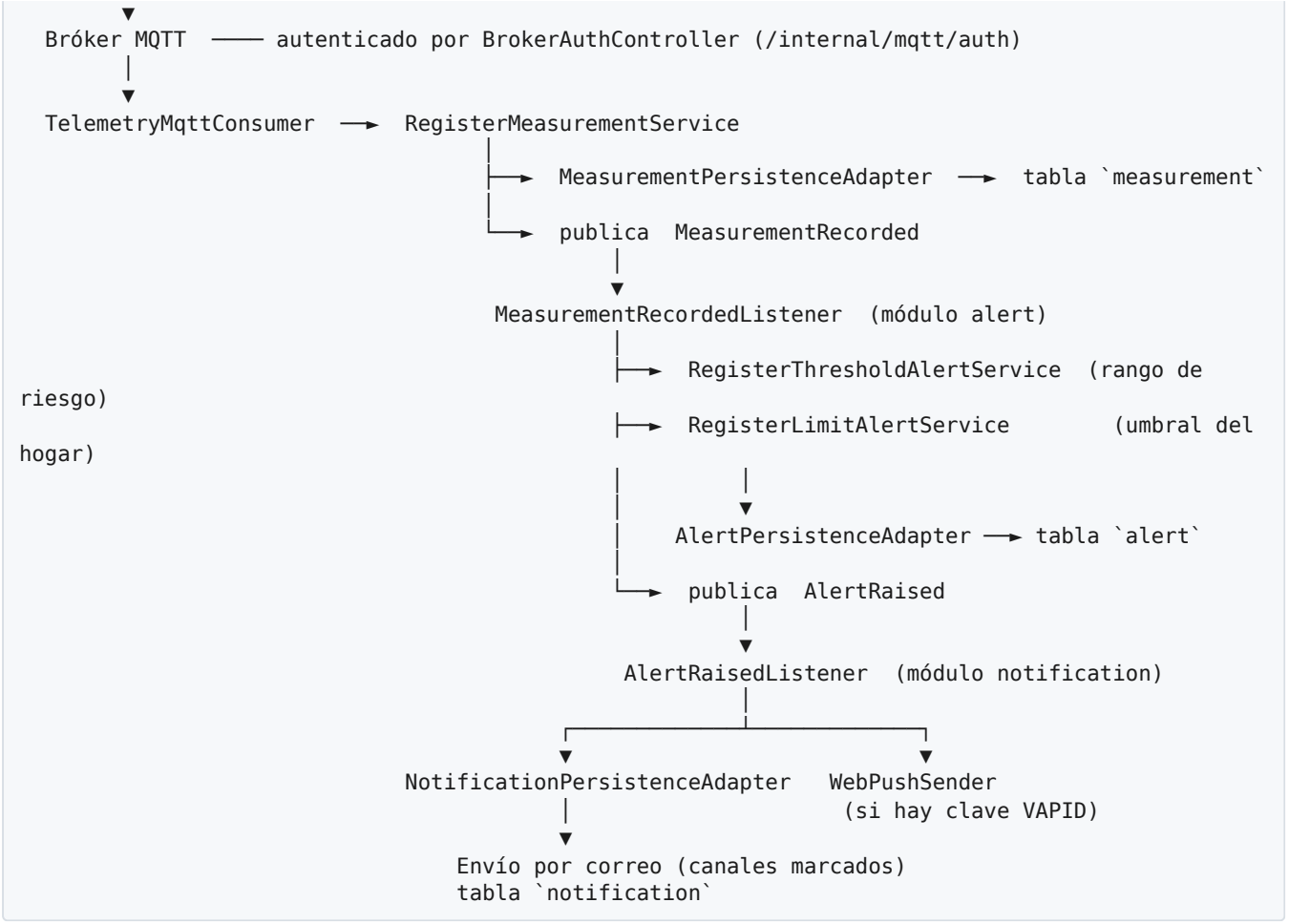
Evento	Publicado por	Escuchado por	Efecto
`MeasurementRecorded`	`measurement`	`alert`	Genera la alerta correspondiente a la medición
`DeviceConnectivityLost`	`device`	`alert`	Genera una alerta de conectividad
`DeviceConnectivityRestored`	`device`	—	Cierra automáticamente las alertas de conectividad del dispositivo
`DeviceLinked` / `DeviceUnlinked`	`device`	—	Notifica el cambio de vinculación
`AlertRaised`	`alert`	`notification`	Envía el aviso por los canales configurados
`RecommendationCreated`	`recommendation`	`notification`	Envía el consejo por los canales configurados
`AccountDeleted`	`security`	`home`	Elimina al usuario de todos sus hogares
`HomeDeleted`	`home`	`device`	Desvincula los dispositivos del hogar, revoca sus claves de API y, tras confirmar la transacción, envía a cada dispositivo la orden MQTT `{"cmd":"unlink"}`
`HomeOwnershipTransferred`	`home`	—	Registra el cambio de propietario; ningún módulo lo consume todavía

> **Recomendaciones sin evento de medición.** El módulo `recommendation` no escucha `MeasurementRecorded`: `RecommendationScheduler` lo activa cada hora (intervalo `recommendation.evaluate-interval`, por defecto `PT1H`, con un retardo inicial de dos minutos) y publica `RecommendationCreated` por cada recomendación generada.

4.2 Flujo completo de una medición

Figura 6. *Cadena reactiva: dispositivo → broker MQTT → medición → alerta → notificación.*

```
  ESP32 (sensor PZEM-004T V3)
        |  publica telemetría cada 60 s
```



4.3 Comunicación asíncrona adicional

Mecanismo	Clase	Frecuencia	Efecto
`@Scheduled`	`DeviceInactivityScheduler`	Cada 30 s	Marca `OFFLINE` los dispositivos que no reportan desde hace más de 150 s
`@Scheduled`	`RecommendationScheduler`	Cada hora (desfase 2 min)	Evalúa las reglas de recomendación sobre todos los hogares
`@Scheduled` + `ApplicationReadyEvent`	`AbandonedDeliverySweeper`	Cada 5 min y al arrancar	Cierra entregas de correo abandonadas hace más de 30 min
`@Async`	`AlertRaisedListener`, `RecommendationCreatedListener`	Bajo demanda	Libera el hilo de la transacción para el envío del aviso
`ThreadPoolExecutor`	`SecurityResetExecutorConfiguration`	Bajo demanda	Procesa la recuperación de contraseña fuera del hilo de petición

5. Vista de despliegue

Figura 7. *Despliegue en el entorno de desarrollo.*

Componente	Tecnología	Puerto	Comunicación
Aplicación web	Vite (estáticos) sobre cualquier servidor	`5173` (desarrollo)	HTTPS/HTTP → API REST
Aplicación móvil	Expo (solo Android)	—	HTTPS/HTTP → API REST
API REST	Spring Boot 4.1.1 sobre Java 21	`8080`	HTTP/1.1, JSON bajo `/api/v1`

Componente	Tecnología	Puerto	Comunicación
Base de datos	MySQL 8.4 en contenedor	`3306`	JDBC con `mysql-connector-j`
Bróker MQTT	Mosquitto	`1883` (interno) · `8883` (dispositivos, TLS)	MQTT 3.1.1 con autenticación y ACL
Servidor de correo	SMTP (Mailpit en desarrollo)	`587` / `1025`	SMTP con STARTTLS

5.1 Imagen de ejecución

Etapa	Imagen base	Contenido
Construcción	`eclipse-temurin:21-jdk`	Descarga de dependencias y empaquetado del JAR
Ejecución	`eclipse-temurin:21-jre`	Usuario de sistema `app` con UID 10001, puerto 8080 exponido, memoria máxima del 75 %

5.2 Configuración por entorno

Variable	Propósito
`MYSQL_PASSWORD` / `MYSQL_ROOT_PASSWORD`	Credenciales de la base de datos
`MAIL_HOST` / `MAIL_PORT` / `MAIL_USERNAME` / `MAIL_PASSWORD` / `MAIL_FROM`	Servidor de correo
`SECURITY_RESET_PEPPER`	Pepper del HMAC de los códigos de recuperación
`SECURITY_CORS_ALLOWED_ORIGINS`	Orígenes permitidos por CORS
`SECURITY_JWT_PRIVATE_KEY_PATH` / `SECURITY_JWT_PUBLIC_KEY_PATH`	Rutas del par de claves RSA
`GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET`	Inicio de sesión con Google
`MQTT_BROKER_URL` / `MQTT_BACKEND_USER` / `MQTT_BACKEND_PASS` / `MQTT_CLIENT_ID`	Bróker MQTT
`DEVICE_MQTT_PUBLIC_HOST` / `DEVICE_MQTT_PUBLIC_PORT`	Datos que se entregan al dispositivo al aprovisionarlo
`NOTIFICATION_WEB_URL` / `NOTIFICATION_VAPID_PUBLIC_KEY` / `NOTIFICATION_VAPID_PRIVATE_KEY`	Notificaciones push del navegador
`VITE_API_BASE_URL`	Origen de la API para la web
`EXPO_PUBLIC_API_BASE_URL`	Origen de la API para la aplicación móvil

> **Los secretos se leen desde el entorno, nunca desde el repositorio.** El archivo `.env` se importa automáticamente mediante `spring.config.import`. Los archivos `.env`, `application-local.yaml` y `*.pem` están declarados en `.gitignore`.

6. Arquitectura de los front-ends

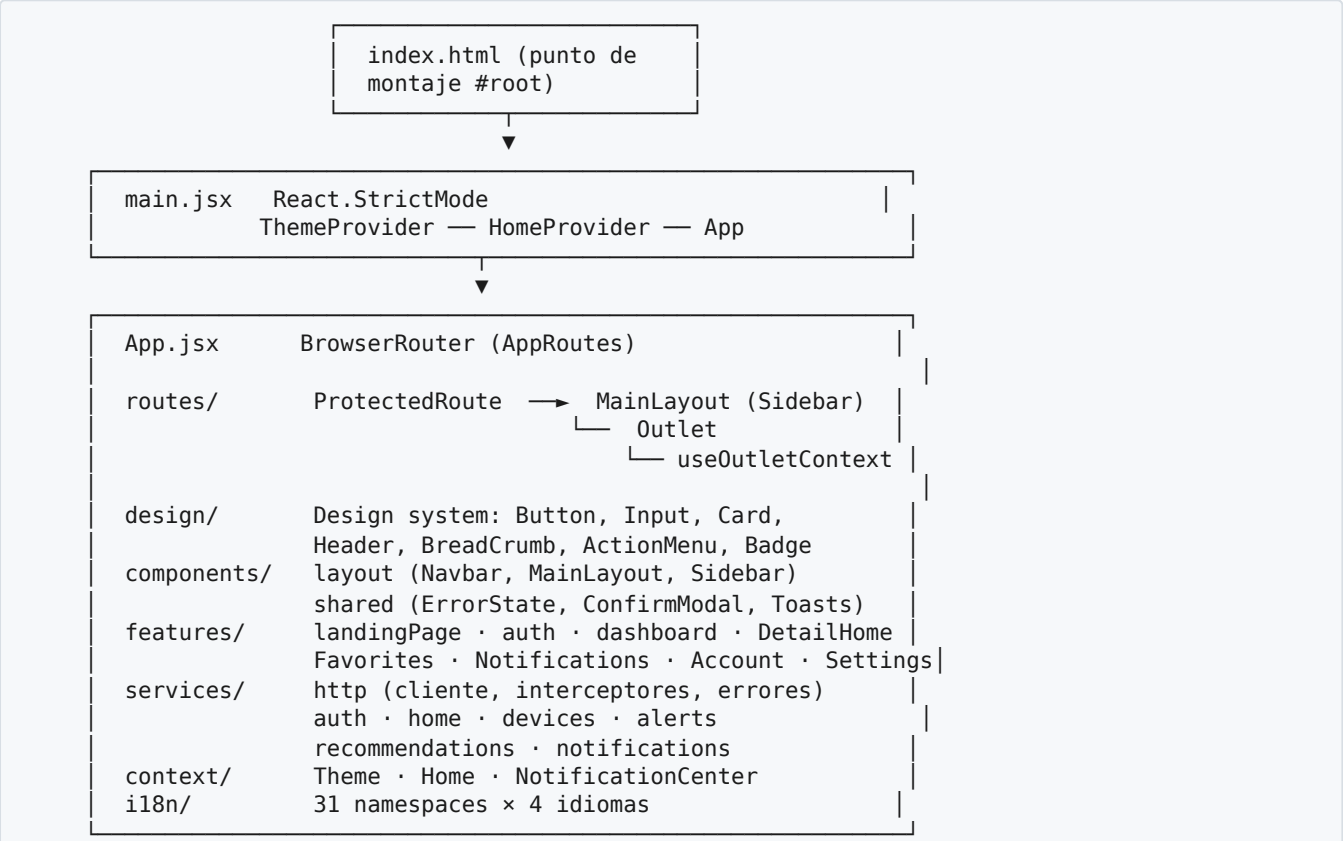
6.1 Principio común

Web y móvil comparten el **mismo modelo de dominio** y el **mismo contrato de API**. Ambos replican, de manera explícita y documentada, la misma paleta de cuatro temas, la misma escala tipográfica y los mismos colores de gráficas.

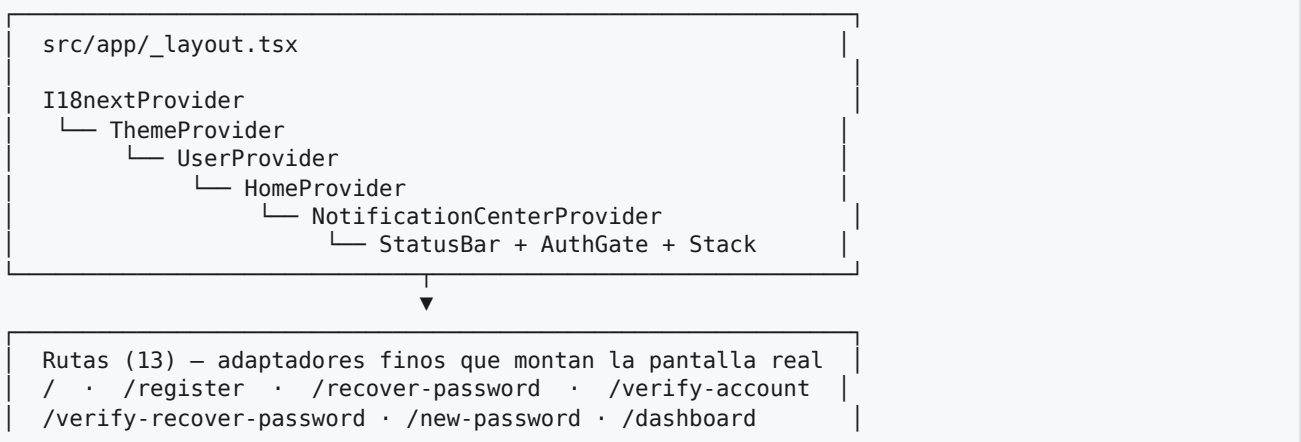
Capa conceptual	Web	Móvil
Presentación	React 19 + Vite 8 (JavaScript)	Expo SDK 57 + React Native 0.86 (TypeScript estricto)

Capa conceptual	Web	Móvil
Navegación	`react-router-dom` 7, rutas anidadas	`expo-router` 57, rutas basadas en archivos
Estado global	React Context (3 contextos)	React Context (4 contextos)
Cliente HTTP	`axios` + interceptores	`axios` + interceptores
Formularios	`react-hook-form` + `zod`	`react-hook-form` + `zod`
Gráficas	`recharts`	`react-native-chart-kit`
Estilos	CSS Modules + variables CSS	`StyleSheet` + sistema de tokens propio
Diseño	Sistema de tokens CSS propio	Sistema de tokens equivalente
Idiomas	4 (`es`, `en`, `fr`, `pt`)	4 (`es`, `en`, `fr`, `pt`)
Persistencia local	`localStorage`	`AsyncStorage`

6.2 Arquitectura de la aplicación web



6.3 Arquitectura de la aplicación móvil



```
│  /homes · /home/[id] · /favorites · /notifications          │
│  /settings · /account                                       │
├─────────────────────────────────────────────────────────────┤
│  components/   Design system propio (18 componentes)        │
│                + layout (Sidebar, BottomTabBar, AuthGate)   │
│  features/     auth · dashboard · homes · favorites         │
│                notifications · settings · account           │
│  services/     http · auth · home · devices · measurement   │
│                alerts · recommendations · notifications     │
│  constants/ + theme/   Tokens y paletas (32 tokens × 4)     │
│  i18n/         25 namespaces × 4 idiomas                    │
└─────────────────────────────────────────────────────────────┘
```

6.4 Decisión de arquitectura móvil

La aplicación móvil usa un **únic Stack plano** de Expo Router, sin pestañas nativas ni apilamientos anidados. La barra de pestañas del detalle del hogar y el menú lateral son **componentes propios** construidos sobre `Animated` y estado local, no rutas del enrutador.

Decisión	Justificación
Stack plano	El producto es «la web equivalente en el teléfono»: un menú lateral con la lista de hogares y una barra inferior de secciones refleja esa estructura
Barra de pestañas propia	Evita que cambiar de pestaña encole una pantalla nueva en la pila de navegación, de modo que el botón «atrás» salga del hogar y no de la sección
Menú lateral propio	Permite un acordeón con la lista de hogares que se sincroniza con el contexto global sin registrar rutas dinámicas

7. Atributos de calidad y cómo los soporta la arquitectura

Atributo	Requisito (especificación de requisitos)	Soporte arquitectónico
Rendimiento	RNF-REN-01 (respuesta < 2 s)	Consultas agregadas en base de datos (`GROUP BY` por hora en UTC); consultas bajo demanda y no en cada render; carga diferida no usada por decisión de alcance
Escalabilidad	RNF-ESC-01 (crecer en usuarios y dispositivos)	Monolito modular con contextos independientes; sin estado en servidor (JWT) permite replicar instancias sin sesión compartida
Seguridad	RNF-SEG-01 a RNF-SEG-10	Autenticación RS256 con par de claves RSA; authorization por RBAC contra el estado vivo; validación en dos capas (Zod + Bean Validation); CORS restrictivo; auditoría de acciones
Usabilidad	RNF-USA-01 (sin nivel formal WCAG declarado)	Sistema de diseño propio con tokens; cuatro temas y cuatro idiomas; estados declarativos (`loading` / `error` / `empty` / `content`); mensajes de error traducidos por contrato
Disponibilidad	RNF-DIS-01 (sin meta fijada), RNF-DIS-06	Actuator `/actuator/health`; entrega de notificaciones con reintento mediante el barrido de abandonadas; modo degradado: una caída de red no cierra la sesión ni vacía la bandeja
Mantenibilidad	RNF-MAN-01 a RNF-MAN-04	Límites de módulo ejecutables; seis capas de pruebas; código de dominio sin dependencias de framework; documentación en los `package-info.java`
Compatibilidad	RNF-COM-01 a RNF-COM-04	El backend se empaqueta en imagen con JRE; la web es una compilación estática; la móvil se compila solo para Android

8. Reglas arquitectónicas autoimuestas

Las siguientes reglas están declaradas en el código y son verificables.

#	Regla	Cómo se verifica
1	El dominio no importa Spring	Revisión de los paquetes `domain/model`; prueba de módulo

#	Regla	Cómo se verifica
2	Ningún módulo importa clases internas de otro	`ModularityTests` (`ApplicationModules.verify()`)
3	Toda consulta a base de datos pasa por un puerto de salida	Los casos de uso no conocen repositorios de Spring Data
4	Las referencias entre contextos son identificadores planos	Ausencia total de anotaciones de asociación JPA
5	Un `@RestControllerAdvice` por módulo	`SecurityExceptionHandler` es global porque también lo requiere la cadena de filtros
6	Los casos de uso reciben comandos y devuelven resultados	Tipos `*Command`, `*Query` y `*Result` en `application/`
7	Las entidades JPA nunca se exponen al exterior	Todos los controladores devuelven DTOs
8	Los identificadores de ruta se codifican en el cliente	`encodeURIComponent` en los dos front-ends
9	El mensaje de error mostrado al usuario nunca proviene del servidor	Catálogo de traducción por operación y código de estado
10	Los fallos de red no vacían datos ya cargados	Contexto de notificaciones de ambos front-ends

9. Contexto de los diagramas de análisis

Este punto de la documentación **no** sustituye al análisis del software. Las relaciones con el punto 03 son las siguientes:

Artefacto de origen	Qué aporta a este punto
Historias de usuario y casos de uso (punto 02)	Definen las funcionalidades que la arquitectura debe soportar
Diagramas de navegación web y móvil	Definen el mapa de pantallas que se implementa en `features/`
Mockups	Definen el aspecto visual esperado de cada pantalla; se contrastan con `05.5-interfaces-graficas.md`

10. Artefactos relacionados

Artefacto	Ubicación
Modelo de datos	`05.2-modelo-de-datos.md`
Modelo de clases	`05.3-modelo-de-clases.md`
Patrones de diseño	`05.4-patrones-de-diseno.md`
Interfaces gráficas	`05.5-interfaces-graficas.md`
Diagramas de arquitectura, clases y MER	`assets/diagramas/`
Manual técnico del proyecto	`10-manual-tecnico/`